

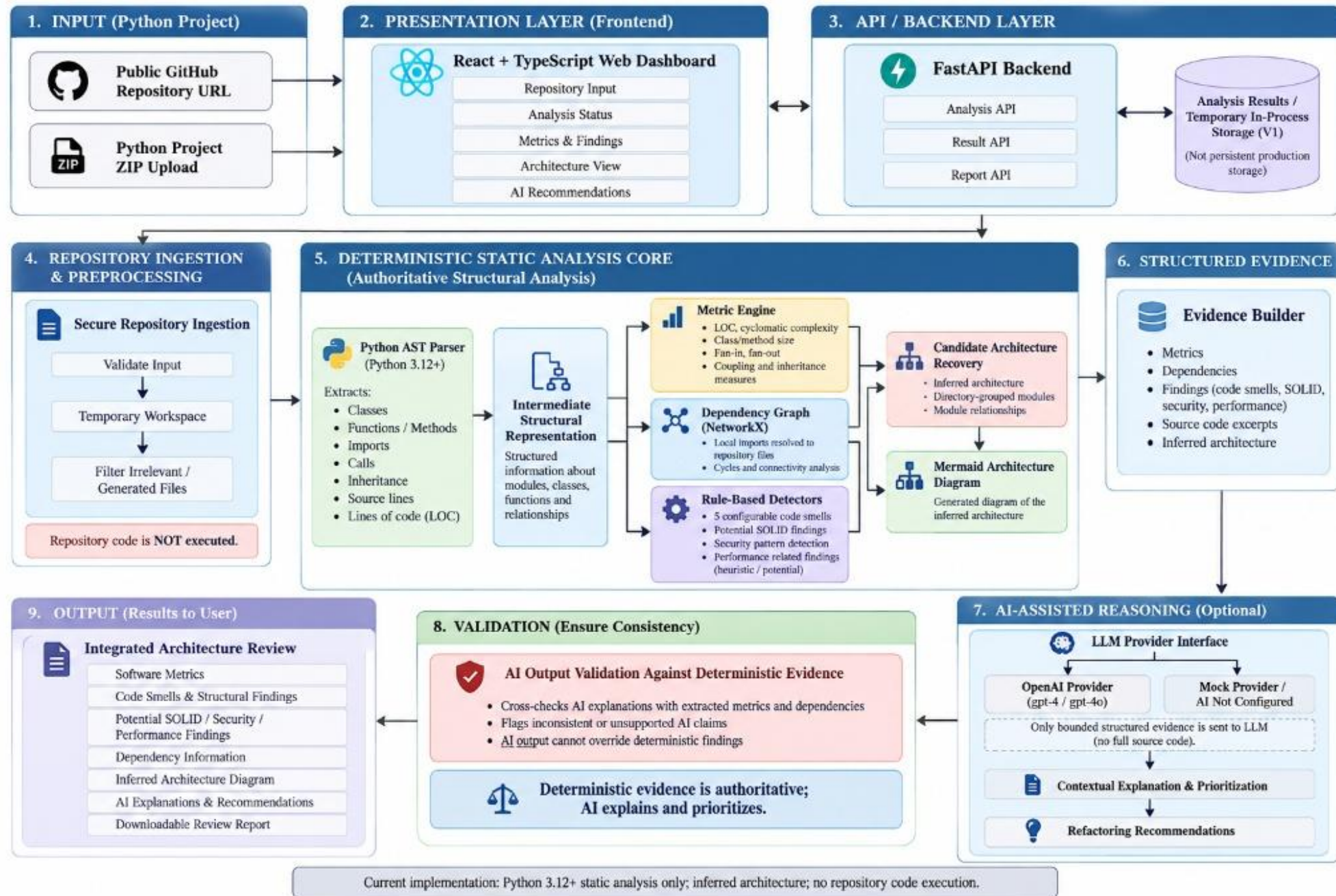
Problem Statement

- Modern software projects become increasingly **complex and difficult to maintain** as they evolve.
- Code smells such as **Long Methods, God Classes, excessive coupling, and cyclic dependencies** can reduce software quality.
- Traditional static-analysis tools mainly identify predefined violations but provide **limited contextual explanation** of their impact.
- Understanding the overall **project structure and dependencies** manually becomes difficult for large repositories.
- LLM-based analysis can provide useful explanations, but relying only on AI may result in **unsupported or inconsistent findings**.
- Hence, there is a need for a **hybrid framework** that combines reliable static analysis with AI-assisted reasoning to identify issues and provide meaningful refactoring recommendations.

Objectives

- To develop an automated framework for analyzing **Python source-code repositories** without executing their code.
- To extract structural information using **Python AST-based static analysis**.
- To calculate software metrics and construct **dependency graphs** for understanding project structure.
- To detect potential **code smells and structural/design-quality issues** using deterministic rules.
- To generate an **inferred structural representation** of the analyzed Python project.
- To integrate an **LLM-based AI review** for contextual explanations and refactoring recommendations.
- To present metrics, findings, dependencies, structural information, and AI recommendations through a **unified web-based dashboard**.

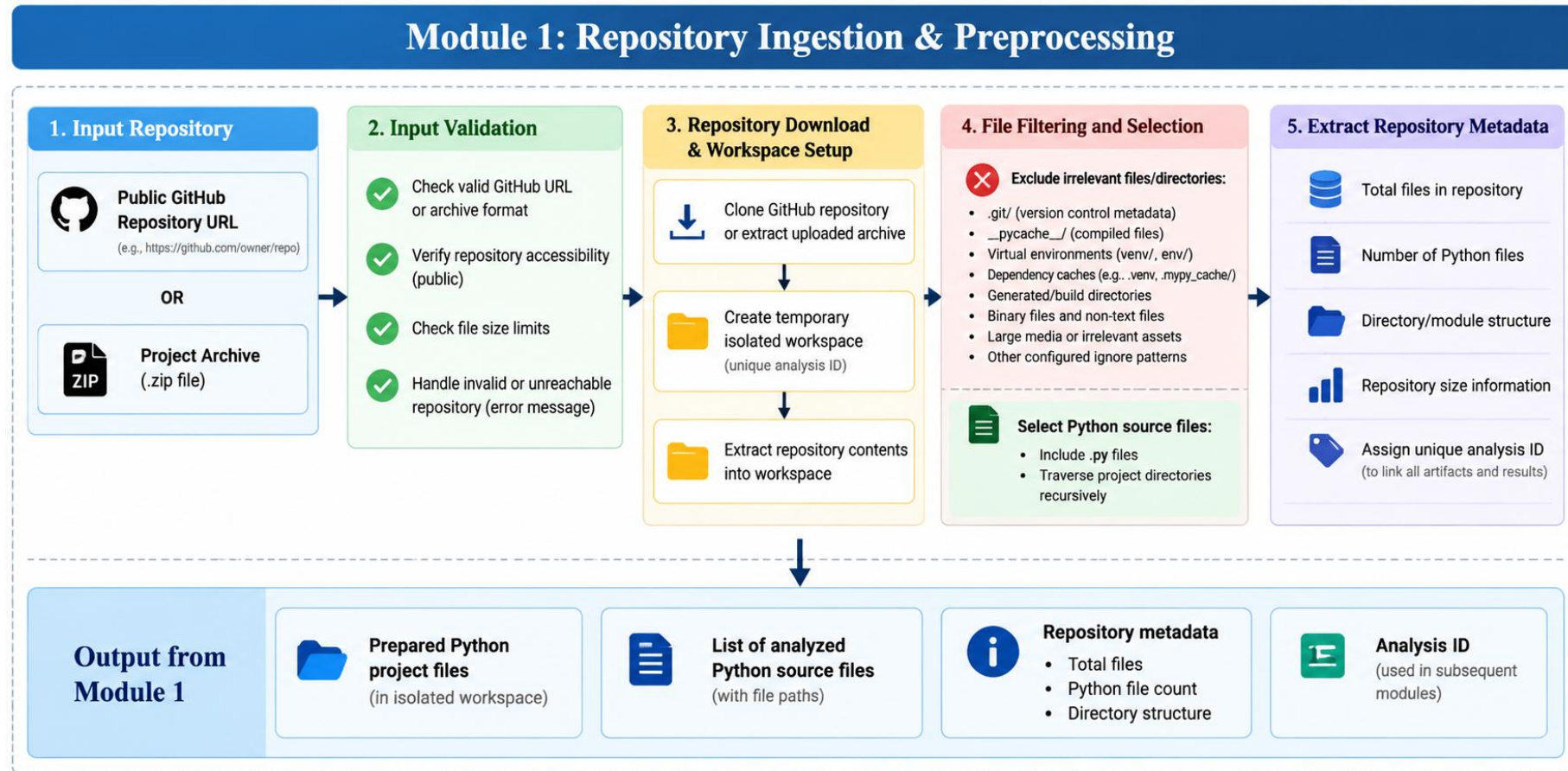
OVERALL ARCHITECTURE DIAGRAM



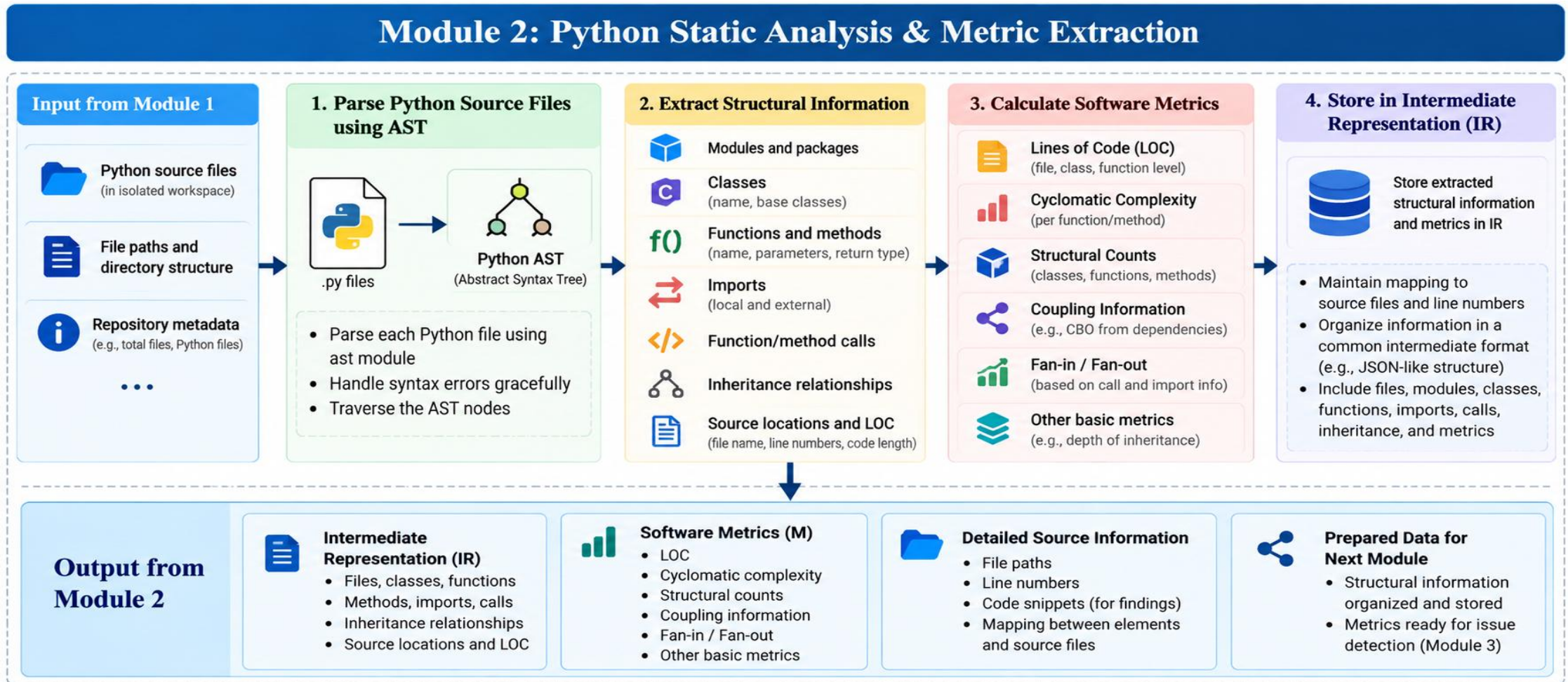
List of Modules

- Module 1: Repository Ingestion & Preprocessing
- Module 2: Python Static Analysis & Metric Extraction
- Module 3: Dependency & Code Quality Analysis
- Module 4: Project Structure & AI-Assisted Review
- Module 5: Integrated Results & Visualization

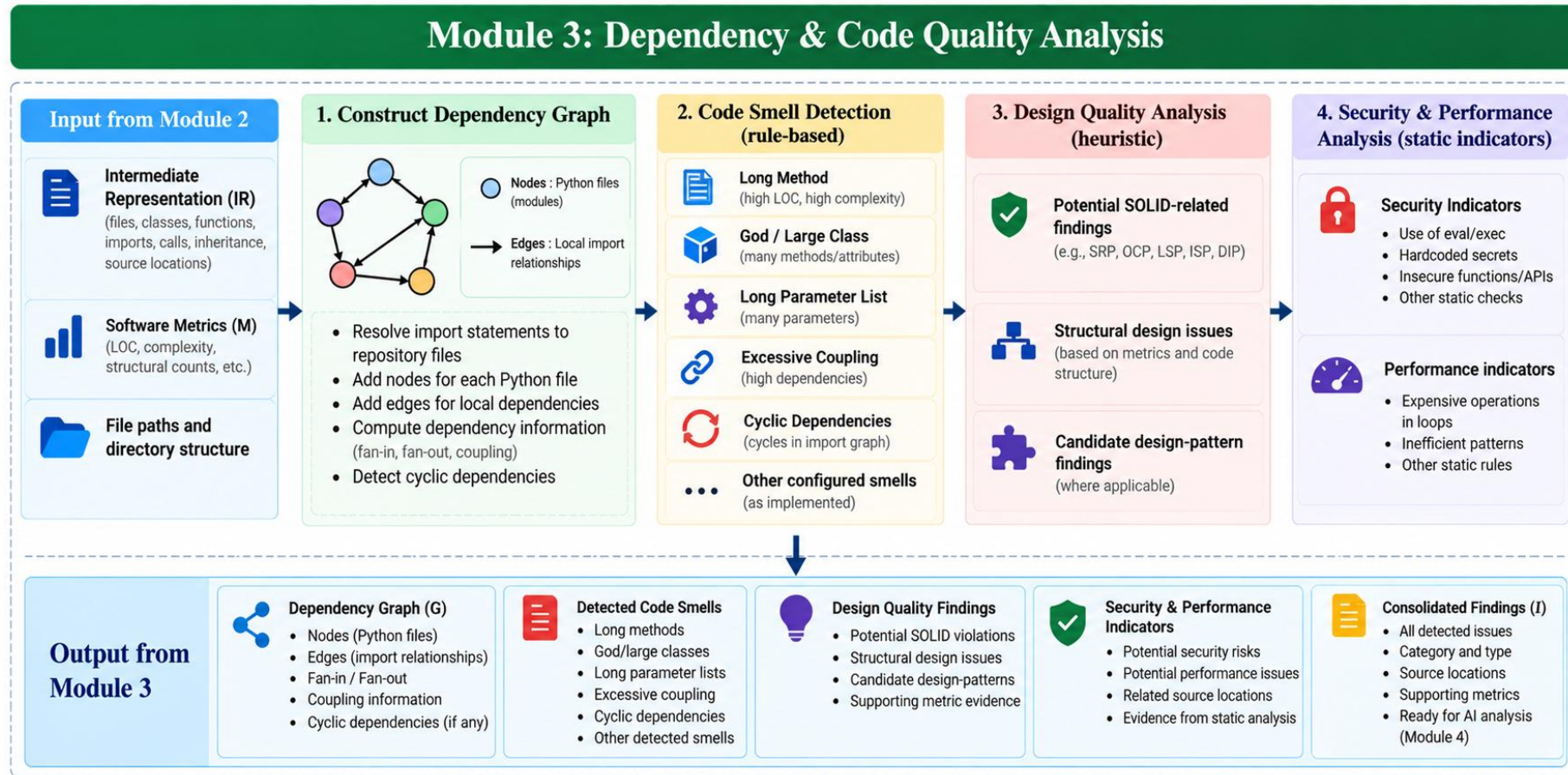
Module 1: Repository Ingestion & Preprocessing



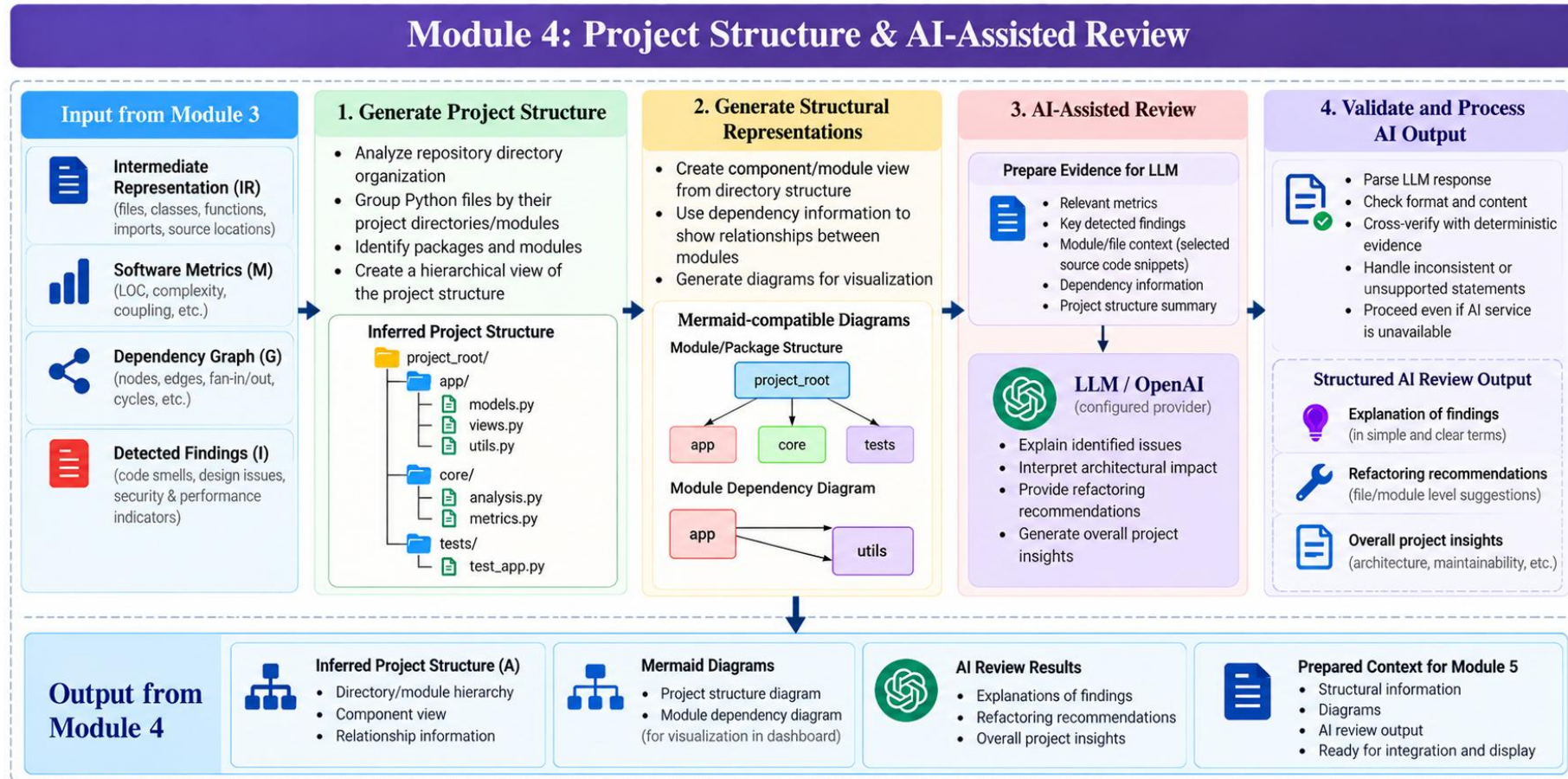
Module 2: Python Static Analysis & Metric Extraction



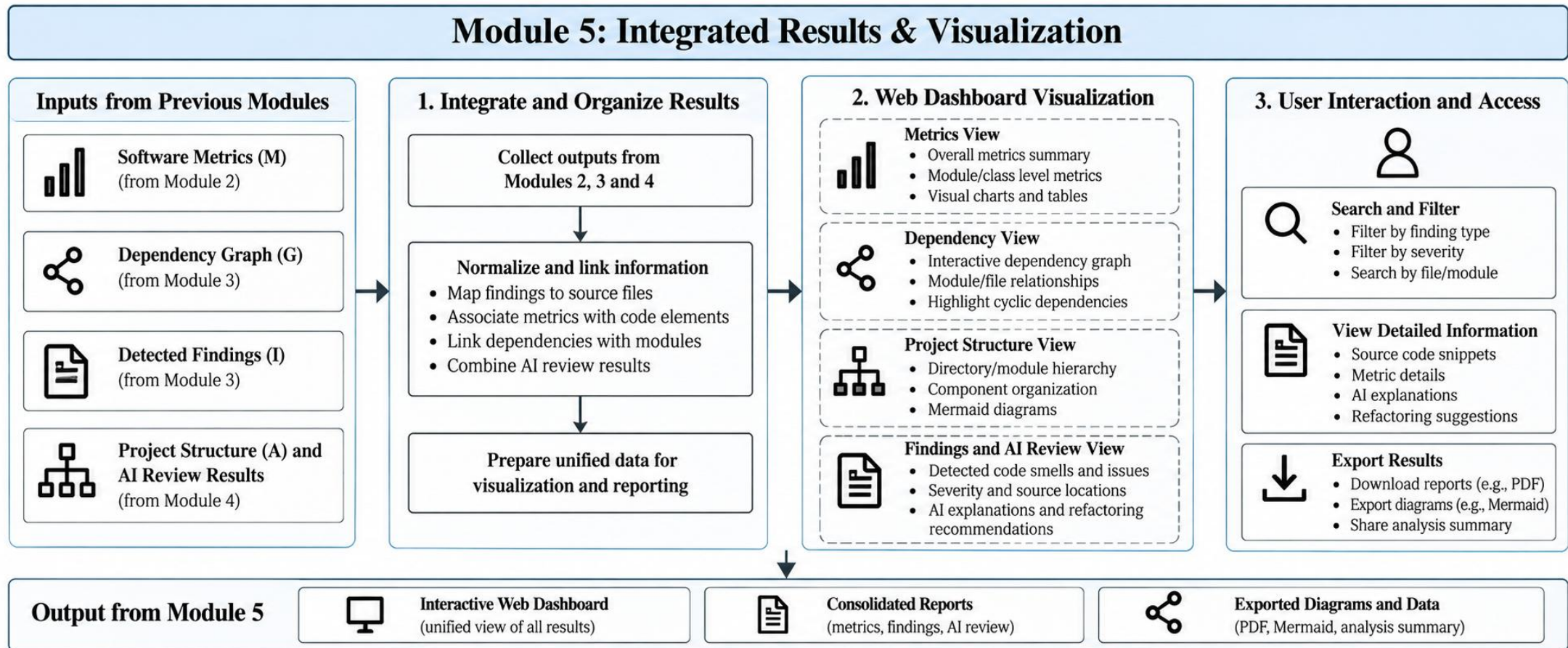
Module 3: Dependency & Code Quality Analysis



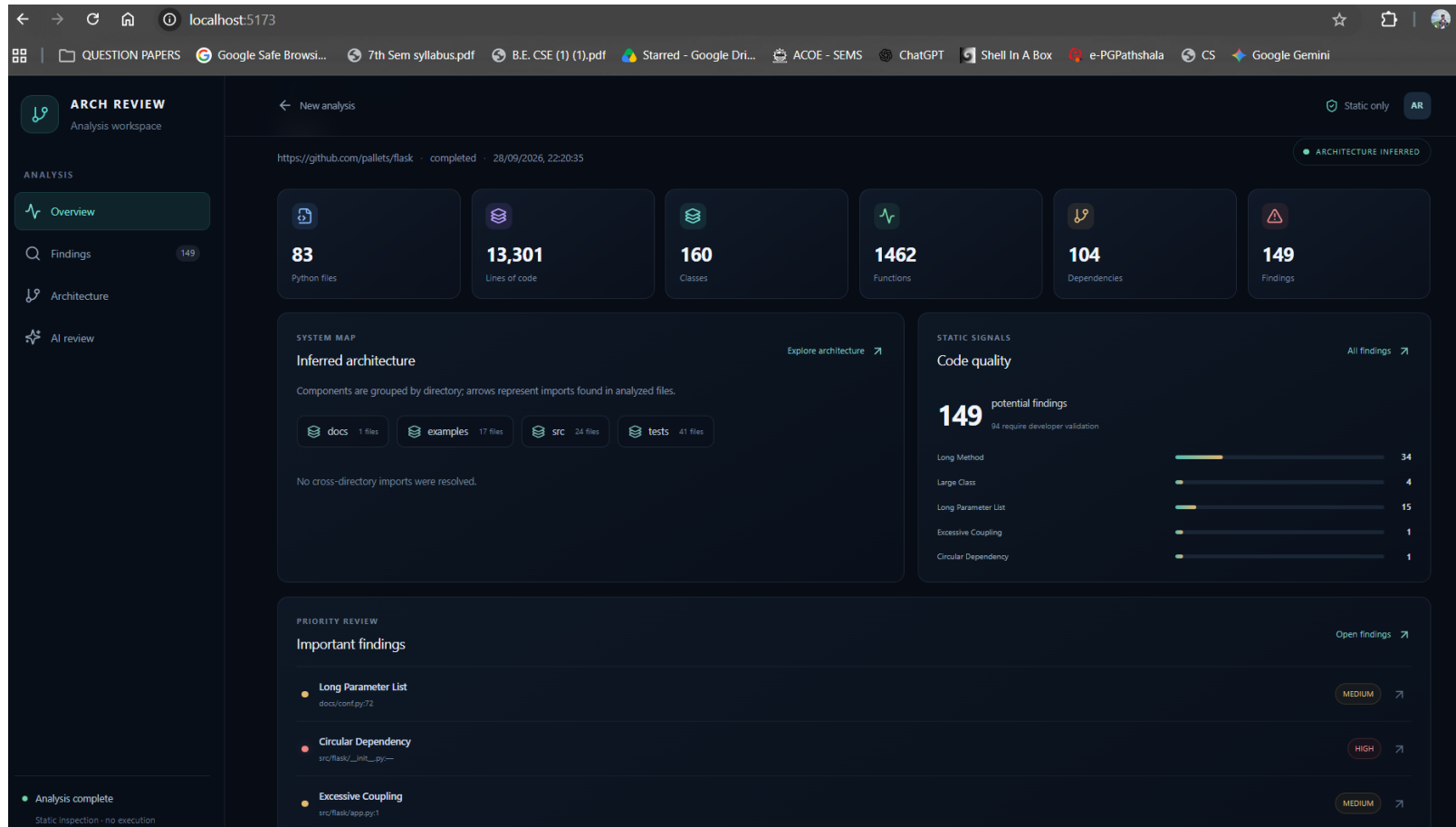
Module 4: Project Structure & AI-Assisted Review



Module 5: Integrated Results & Visualization



Dashboard



Architecture Inferred Findings

The screenshot displays the ARCH REVIEW tool interface, which is a static analysis workspace. The left sidebar contains navigation options: Overview, Findings (149), Architecture, and AI review. The main panel shows the 'Findings' section for a project named 'pallets/flask'. The findings are categorized by severity and type, with a search bar and filters available. The findings list includes:

- Long Parameter List** (Medium severity, docs/conf.py:72)
- Circular Dependency** (High severity, src/flask/_input.py:—)
- Excessive Coupling** (Medium severity, src/flask/app.py:1)
- Large Class** (High severity, src/flask/app.py:110)
- Potential ISP concern** (Low severity, src/flask/app.py:110)
- Potential LSP concern** (Low severity, src/flask/app.py:110)
- Potential OCP concern** (Low severity, src/flask/app.py:110)
- Potential SRP concern** (Low severity, src/flask/app.py:110)

The interface also shows a status bar at the bottom indicating 'Analysis complete' and 'Static inspection - no execution'.

Inferred Architecture – Candidate Components

The screenshot displays the ARCH REVIEW web application interface. The browser address bar shows 'localhost:5173'. The application has a dark theme and a sidebar on the left with the following menu items: 'ARCH REVIEW Analysis workspace', 'ANALYSIS', 'Overview', 'Findings' (with a badge showing 149), 'Architecture' (highlighted), and 'AI review'. The main content area is titled 'Architecture' and shows the path 'ANALYSIS / ARCHITECTURE'. Below this, it indicates the source is 'https://github.com/pallets/flask' and the analysis is 'completed · 28/09/2026, 22:20:35'. A button labeled 'ARCHITECTURE INFERRED' is visible. The main section is titled 'Candidate components' and includes a disclaimer: 'The structure below is inferred from file paths and local import edges. It may not match the architecture intended by the project authors.' Below this, four panels show candidate components categorized by directory:

- docs** (1 source files):
 - docs/conf.py
- examples** (17 source files):
 - examples/celery/make_celery.py
 - examples/celery/src/task_app/__init__.py
 - examples/celery/src/task_app/tasks.py
 - examples/celery/src/task_app/views.py
 - examples/javascript/js_example/__init__.py
 - examples/javascript/js_example/views.py
 - examples/javascript/tests/conftest.py
 - examples/javascript/tests/test_js_example.py
 - and 9 more
- src** (24 source files):
 - src/flask/__init__.py
 - src/flask/__main__.py
 - src/flask/app.py
 - src/flask/blueprints.py
 - src/flask/cli.py
 - src/flask/config.py
 - src/flask/ctx.py
 - src/flask/debughelpers.py
 - and 16 more
- tests** (41 source files):
 - tests/conftest.py
 - tests/test_appctx.py
 - tests/test_apps/blueprintapp/__init__.py
 - tests/test_apps/blueprintapp/apps/__init__.py
 - tests/test_apps/blueprintapp/apps/admin/__init__.py
 - tests/test_apps/blueprintapp/apps/frontend/__init__.py
 - tests/test_apps/cliapp/__init__.py
 - tests/test_apps/cliapp/app.py
 - and 33 more

Component Relationships and Mermaid Diagram

The screenshot displays the ARCH REVIEW Analysis workspace interface. The left sidebar contains navigation options: Overview, Findings (485), Architecture (selected), and AI review. The main content area is titled 'New analysis' and shows the following sections:

- LOCAL IMPORTS**
 - Component relationships**

Source	Target	Edges
docs_src	fastapi	446 file edges
tests	docs_src	95 file edges
tests	fastapi	732 file edges
tests	scripts	2 file edges
- GENERATED FROM THE SAME GRAPH**
 - Mermaid diagram**

```
graph TD
    component_0["docs_src"]
    component_1["fastapi"]
    component_2["scripts"]
    component_3["tests"]
    component_0 --> component_1
    component_3 --> component_0
    component_3 --> component_1
    component_3 --> component_2
```

This is Mermaid source generated from the analyzed dependency graph.
- CYCLE CHECK**
 - Dependency cycles**

```
fastapi/_init_.py --> fastapi/_compat/_init_.py --> fastapi/_compat/v2.py --> fastapi/applications.py --> fastapi/datastructures.py --> fastapi/dependencies/models.py --> fastapi/dependencies/utils.py --> fastapi/encoders.py --> fastapi/exception_handlers.py --> fastapi/openapi/docs.py --> fastapi/openapi/models.py --> fastapi/openapi/utils.py --> fastapi/param_functions.py --> fastapi/params.py --> fastapi/routing.py --> fastapi/security/base.py --> fastapi/security/oauth2.py --> fastapi/utils.py
```

At the bottom left, a status bar indicates: Analysis complete, Static inspection, no execution.

Hybrid AI - review

The screenshot displays the ARCH REVIEW web application interface, which is used for static analysis and AI review of code repositories. The interface is dark-themed and includes a sidebar on the left with navigation options: Overview, Findings (153), Architecture, and AI review (selected). The main content area shows the results of an AI review for the repository `https://github.com/pallets/flask`, completed on 28/9/2026 at 1:49:28 am. The review status is "ARCHITECTURE INFERRED".

AI review

`https://github.com/pallets/flask` · completed · 28/9/2026, 1:49:28 am

OPTIONAL AI INTERPRETATION · COMPLETED

The repository under review is Flask, a popular lightweight WSGI web microframework for Python developed by Pallets. The provided static analysis identifies 10 deterministic findings across circular dependencies, large classes, and long methods.

ARCHITECTURE CONTEXT

AI summary

Flask maintains a structured modular design split between sans-io core modules, application logic, and helper utilities. However, due to its framework nature and centralized design, certain core classes like Flask and App accumulate many responsibilities, resulting in high method counts and long execution blocks. Additionally, cyclic imports are common in Python web frameworks where core objects, helpers, and configurations must tightly reference each other.

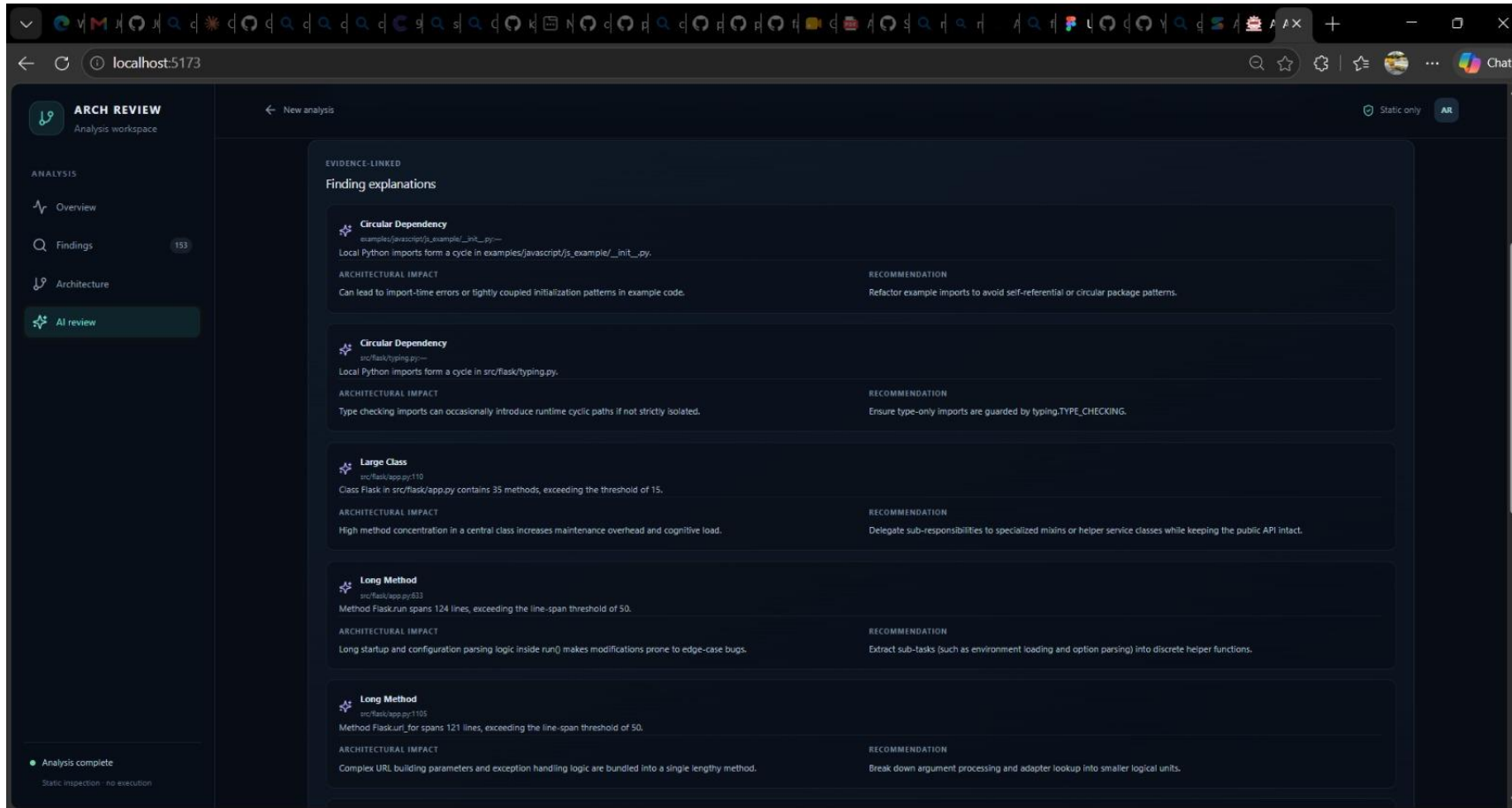
EVIDENCE-LINKED

Finding explanations

Finding	Architectural Impact	Recommendation
Circular Dependency example/javascripts/example/_init.py Local Python imports form a cycle in example/javascripts/example/_init.py.	Can lead to import-time errors or tightly coupled initialization patterns in example code.	Refactor example imports to avoid self-referential or circular package patterns.
Circular Dependency src/flask/typing.py Local Python imports form a cycle in src/flask/typing.py.	Type checking imports can occasionally introduce runtime cyclic paths if not strictly isolated.	Ensure type-only imports are guarded by <code>typing.TYPE_CHECKING</code> .

Analysis complete
Static inspection · no execution

Hybrid AI – review contrn.



The screenshot displays the ARCH REVIEW Analysis workspace in a web browser. The interface is dark-themed and includes a sidebar on the left with navigation options: Overview, Findings (153), Architecture, and AI review (selected). The main content area, titled 'New analysis', shows a list of findings under the heading 'EVIDENCE-LINKED Finding explanations'. Each finding entry includes a star icon, a title, a code path, a description, an architectural impact, and a recommendation.

Finding	Code Path	Description	Architectural Impact	Recommendation
Circular Dependency	examples/javascript/js_example/_init.py	Local Python imports form a cycle in examples/javascript/js_example/_init.py.	Can lead to import-time errors or tightly coupled initialization patterns in example code.	Refactor example imports to avoid self-referential or circular package patterns.
Circular Dependency	src/fask/typing.py	Local Python imports form a cycle in src/fask/typing.py.	Type checking imports can occasionally introduce runtime cyclic paths if not strictly isolated.	Ensure type-only imports are guarded by typing.TYPE_CHECKING.
Large Class	src/fask/app.py:110	Class Flask in src/fask/app.py contains 35 methods, exceeding the threshold of 15.	High method concentration in a central class increases maintenance overhead and cognitive load.	Delegate sub-responsibilities to specialized mixins or helper service classes while keeping the public API intact.
Long Method	src/fask/app.py:833	Method Flask.run spans 124 lines, exceeding the line-span threshold of 50.	Long startup and configuration parsing logic inside run() makes modifications prone to edge-case bugs.	Extract sub-tasks (such as environment loading and option parsing) into discrete helper functions.
Long Method	src/fask/app.py:1105	Method Flask.url_for spans 121 lines, exceeding the line-span threshold of 50.	Complex URL building parameters and exception handling logic are bundled into a single lengthy method.	Break down argument processing and adapter lookup into smaller logical units.

At the bottom left, a status bar indicates 'Analysis complete' and 'Static inspection - no execution'.